

Disclaimer

This is by no means a production ready code! The main purpose of this code is demonstrating that the splean solver approach (explained and evaluated in detail [here](#)) can be turned into reality. It is therefore mostly for educational purpose and so I tried to comment the source code extensively within the files. Otherwise, the code should be pretty self-explanatory.

Files

In this section I will give a brief summary about each source code file and its contents.

splean.cpp

This file contains the start up and skeleton code. The main function parses the command line arguments. Set up the threads and reserves all the memory (in line 435). It starts a solving attempt by calling the function "solve" on each thread. For better comparison, I kept the input and output format very close to the [lean solver approach from John Tromp](#).

The solve function sets up the data structure and runs for each batch the "trim", "walk" and, if cycle candidates are found, the "check" function.

These functions do what their names suggest. They perform the "trim" and "walk" phases of the algorithm as well as checking for cycles.

splean.hpp

This is the general header file. It contains the solver's main parameter (most of them can also be controlled by passing compiler arguments). If arguments are not explicitly set, it tries to come up with reasonable estimates.

It also contains the B(readth) f(irst) s(earch) tree structure. This structure is used to check if indeed a cycle was found after the "walk" step is completed. Ideally, only a small amount of candidate anchor edges are found at the end of the "walk" step. So, the data structure can be quite small. However, it still needs to maintain a full history while "walk"-ing the full path again, to later successfully retrieve all edges that constitute the cycle. It does this, by recording each edge and maintaining a tree structure where each node (edge) knows its parent (previous edge). It also maintains in a multi-hashmap all nodes' current leaf edges, to decide if a potential new edge is actually incident to a leaf node and extends the path/tree further. It also contains a function called "prune" which prunes the edges of the tree. If a path leads to a dead end, all stored edges get removed from the data structure to save space. However, this also costs time and pruning can be deactivated by setting the BFS_LINK_BITS parameter to 32.

hashmap.hpp

This file contains the implementation of a simple hash-based multimap with linear probing as open addressing method. However, it allows for an amount of slots that is not a power of two. This is crucial for memory efficiency. The hashmap is used by the BFS-tree data structure to maintain a list of all current leaf-/end-nodes and their edges.

bloom.hpp

This file contains the implementation of a blocked Bloom filter. It allows an amount of buckets that is not a power of two, which is crucial for memory efficiency. It uses SIMD to speed up the operations as much as possible and the blocks are sized and used in a very cache friendly way.

rice.hpp

This file contains the implementation of a Golomb/Rice compression stream. It allows to store a stream of strictly increasing values efficiently. The implementation is kept pretty simple.

siphash.hpp

The code in this file is based on the file "[siphashxN.h](#)" from [John Tromp](#). If available, it uses SIMD instructions to calculate Siphash in parallel. Currently, it supports 2x (SSE2), 4x (SSE2/AVX2), 8x (AVX2) parallel Siphash calculations.

helper.hpp

This file contains some helper function to create Siphash keys and construct the header, as well as a precise timestamp function for time calculations later; and some functions to "siphash"-ing of edges in batches (consecutive and from an edge stream). It also contains a function to verify a Cuckatoo cycle, [closely adapted from from John Tromp](#) to avoid mistakes.

other files

The other files are adapted from [John Tromp's implementation](#) and they provide functions to calculate blake2b hashes and endianness portability.